

CS 486/586

Chapter 1.2 Data Basics

Credit and thank you go to Professor R.Jesse Chaney from Portland State University for generously providing me with this material.

Relational Databases Consist of Tables

Account

Number	Owner	Balance	Type
101	Smith	\$1,000	checking
102	Wei	\$2,000	checking
103	Smith	\$5,000	savings
104	Jones	\$1,000	checking
105	Martin	\$10,000	checking

Table name

Columns, not fields.

books					
id	title	pagecount	genre	authorid	pubid
1	It	1138	Horror	10	100
2	Hamlet	500	Tragedy	13	103
3	I Know Why the Caged Bird Sings	304	Autobiographical	14	102
4	A Suitable Boy	1349	Drama/Romance	15	103
5	The Joy Luck Club	288	Drama	16	104

Column names

A **primary key** uniquely identifies a row in a table.

Rows, not records

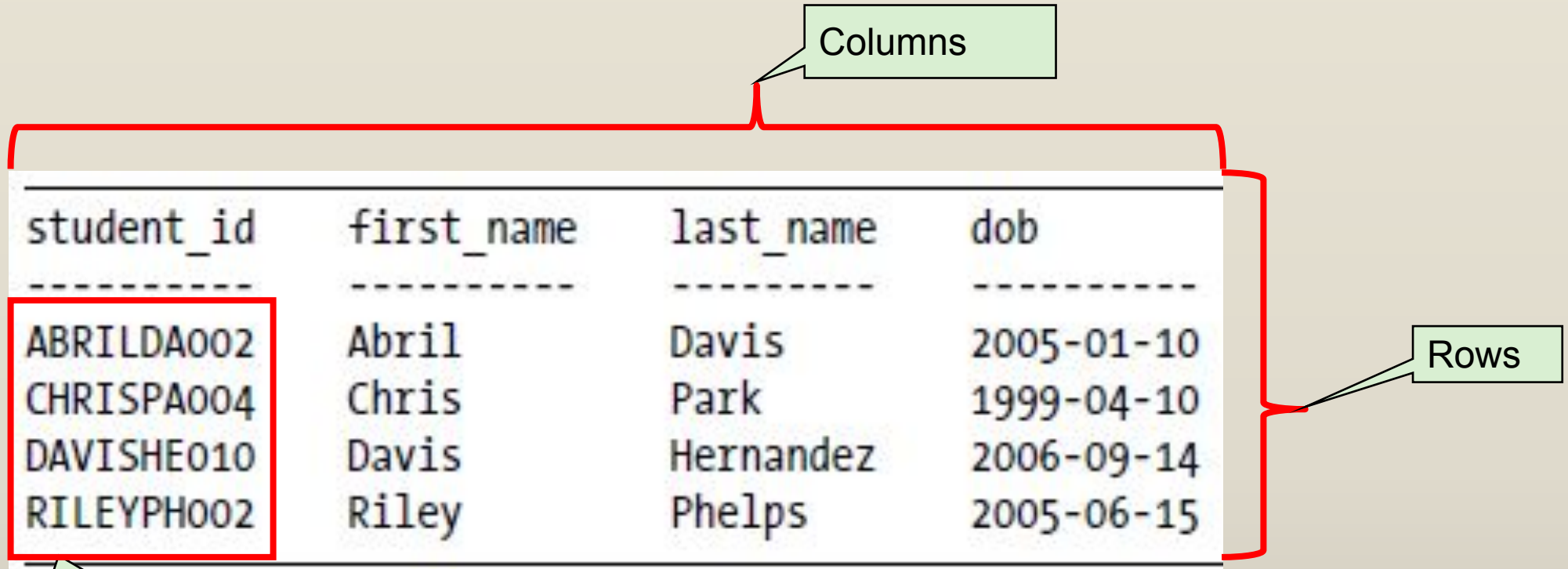
Columns

student_id	class_id	class_section	semester
-----	-----	-----	-----
CHRISPA004	COMPSCI101	3	Fall 2023
DAVISHE010	COMPSCI101	3	Fall 2023
ABRILDA002	ENG101	40	Fall 2023
DAVISHE010	ENG101	40	Fall 2023
RILEYPH002	ENG101	40	Fall 2023

Rows

The table **student_enrollment**, shows the students that are signed up for each class section.

Notice that at least 1 student is signed up for more than 1 class (**DAVISHE010**)



The diagram shows a database table with four columns: student_id, first_name, last_name, and dob. The first four rows of data are highlighted with a red box. A green callout box labeled 'Columns' points to the header row. Another green callout box labeled 'Rows' points to the first four data rows. A third green callout box points to the first column, stating that it is a primary key.

student_id	first_name	last_name	dob
ABRILDA002	Abril	Davis	2005-01-10
CHRISPA004	Chris	Park	1999-04-10
DAVISHE010	Davis	Hernandez	2006-09-14
RILEYPH002	Riley	Phelps	2005-06-15

A **primary key** uniquely identifies a row in a table.

The table **students**, shows the students' names and id. Here, the **student_id** is unique in the table.

Degree or **arity** of a table is the **number of columns**

books

id	title	pagecount	genre	authorid	pubid
1	It	1138	Horror	10	100
2	Hamlet	500	Tragedy	13	103
3	I Know Why the Caged Bird Sings	304	Autobiographical	14	102
4	A Suitable Boy	1349	Drama/Romance	15	103
5	The Joy Luck Club	288	Drama	16	104

Cardinality of a table is the **number of rows** in the current instance.

Tables can be **related**.
The **primary key** in one table is **related to a foreign key in another table**.

authors		
id	name	age
16	Amy Tan	69
17	Laura Esquivel	70

publishers	
id	name
104	Putnam
105	Perfection Learning

Primary key to foreign key relationship

Primary key to foreign key relationship

books					
i d	title	pagecount	genre	authorid	pubid
5	The Joy Luck Club	288	Drama	16	104
6	Like Water for Chocolate	256	Romance/ Tragedy	17	105



Are SQL Keywords Case-Sensitive?

Keywords in **SQL are case-insensitive** for the most popular DBMSs. The database doesn't care whether you write SELECT, select, or sELeCt or SeLeCt or SElect; so, in theory, you can write however you like.

I have not found a database system where the SQL keywords are case sensitive.

Are SQL Table and Column Names Case-Sensitive?

In **PostgreSQL** table names are **case-insensitive**. This behavior doesn't depend on the operating system.

MySQL, by default, **depends on the operating system and its case sensitivity**. This means MySQL is case-insensitive in Windows and macOS, while it is case-sensitive in most Linux systems.

MS SQL Server, by default, **is case-insensitive**.



<https://learnsql.com/blog/sql-case-sensitive/>

Notice that SQL is, by default, **case insensitive**. You can typically configure a database instance to be case sensitive. **Your life will be miserable after that.**

```
CrEaTe ScHeMa If NoT ExIsTs teachers;
```

```
-- Listing 2-2: Creating a table named teachers with six columns
```

```
CREATE TABLE teachers.teachers (  
    id integer generated by default as identity primary key  
    , first_name varchar(25)  
    , last_name varchar(50)  
    , school varchar(50)  
    , hire_date date  
    , salary numeric  
);
```

```
-- This command will remove (drop) the table.  
-- DROP TABLE analysis.teachers;
```

-- Listing 2-3 Inserting data into the teachers table

```
INSERT INTO teachers.teachers (  
    first_name, last_name, school, hire_date, salary)  
VALUES  
    ('Janet', 'Smith', 'F.D. Roosevelt HS', '2011-10-30', 36200)  
    , ('Lee', 'Reynolds', 'F.D. Roosevelt HS', '1993-05-22', 65000)  
    , ('Samuel', 'Cole', 'Myers Middle School', '2005-08-01', 43500)  
    , ('Samantha', 'Bush', 'Myers Middle School', '2011-10-30', 36200)  
    , ('Betty', 'Diaz', 'Myers Middle School', '2005-08-30', 43500)  
    , ('Kathleen', 'Roush', 'F.D. Roosevelt HS', '2010-10-22', 38500)  
;
```

-- drop schema if exists CASCADE;

DML and DDL

SQL DML statements

SELECT
INSERT
UPDATE
DELETE

} Data **M**anipulation **L**anguage

Sometimes, `select` statements are not considered **DML**. I consider them DML.

SQL DDL statements

CREATE DATABASE, TABLE, INDEX
ALTER TABLE, INDEX
DROP DATABASE, TABLE, INDEX

} Data **D**efinition **L**anguage

This is used to create or remove **objects** from the database. Objects are tables, indexes, and such.

Database **objects** are things like tables, indexes, views, entire databases,

...

Natural Key vs Surrogate Key

A **natural key** (also known as business key or domain key) is a type of unique key in a database formed of attributes that exist and are used in the external world outside the database (i.e. in the business domain or domain of discourse)

A **surrogate key** (or synthetic key, pseudokey, entity identifier, factless key, or technical key) in a database is a unique identifier for either an entity in the modeled world or an object in the database. The surrogate key is not derived from application data, unlike a natural (or business) key.

Natural Key vs Surrogate Key

The **advantages** of using a **natural key** to uniquely identify records in a relation include less disk space usage, the **natural key** is an attribute that is related to the business or the real world so in most cases, it is already being stored in the relation.

Another **advantage** of using **natural keys** is that it simplifies enforcement of data quality, and they are easier to relate to real life while designing the database system. They simplify the quality of data as using a **natural key** that is unique in the real world ensures that there cannot be multiple records with the same primary key. Comparing the database schema to a real world scenario is a huge part of designing a database schema and when a natural key is being used in the tables of the database, it makes it easy for the database engineer to engineer the database system.

Natural Key vs Surrogate Key

Use of **natural keys** as unique identifiers in a table has **one main disadvantage**, change of business rules or the change of rules of the attribute in the real world. The definition of the structure of the **natural key** attribute might change in the future.

Natural Key vs Surrogate Key

Advantages of **Surrogate Keys**:

Stability

Surrogate keys typically do not change while the row exists. This has the following advantages:

- Applications cannot lose their reference to a row in the database (since the identifier does not change).
- The natural key data can always be modified, even with databases that do not support cascading updates across related foreign keys.

Requirement changes

Attributes that uniquely identify an entity might change, which might invalidate the suitability of **natural keys**.

Performance

Surrogate keys tend to be a compact data type, such as a four-byte integer. This allows the database to query the single key column faster than it could multiple columns.

Natural Key vs Surrogate Key

Advantages of **Surrogate Keys**:

Compatibility

While using several database application development systems, drivers, and object-relational mapping systems, such as Ruby on Rails or Hibernate, it is much easier to use an integer or GUID **surrogate keys** for every table instead of natural keys in order to support database-system-agnostic operations and object-to-row mapping.

Uniformity

When every table has a uniform **surrogate key**, some tasks can be easily automated by writing the code in a table-independent way.

Validation

It is possible to design **surrogate key** values that follow a well-known pattern or structure which can be automatically verified.

```
CREATE TABLE teachers.teachers (  
    id integer generated by default as identity primary key  
    , first_name varchar(25)  
    , last_name varchar(50)  
    , school varchar(50)  
    , hire_date date  
    , salary numeric  
);
```

CrEaTe ScHeMa If NoT Ex:

-- Listing 2-2: Creating a table named teachers with six columns

```
CREATE TABLE teachers.teachers_nk (  
    first_name varchar(25)  
    , last_name varchar(50)  
    , school varchar(50)  
    , hire_date date  
    , salary numeric  
    , primary key(first_name, last_name)  
);
```

Creating the same table, but using a natural key instead of a surrogate key.

Notice that the pk is now a composite key.

*-- This command will remove (drop) the table.
-- DROP TABLE analysis.teachers;*

Terms to know about database tables

- **Relational database**
- **Table** – made up of **rows** and **columns**
- **Cell** – an individual column value in a row
- **Primary key** – **uniquely identifies a row in a table, cannot be NULL**
- **Composite key** – a **multi-column key**.
- **Non-primary key (unique key)** – can contain a NULL value
- **Index** – used to speed up common queries

A **primary key** has 2 constraints

unique

non-NULL

A **non-primary key (aka unique key)** has 1 constraint:

unique

Terms to know about table relationships

- **Foreign key** – used to establish a relationship between tables
- **Referential integrity** – maintaining valid data references

Relationships

- **One-to-many relationship**
- **One-to-one relationship**
- **Many-to-many relationship**

This is the most common type of foreign key relationship.

- **Natural Key**
- **Surrogate Key**

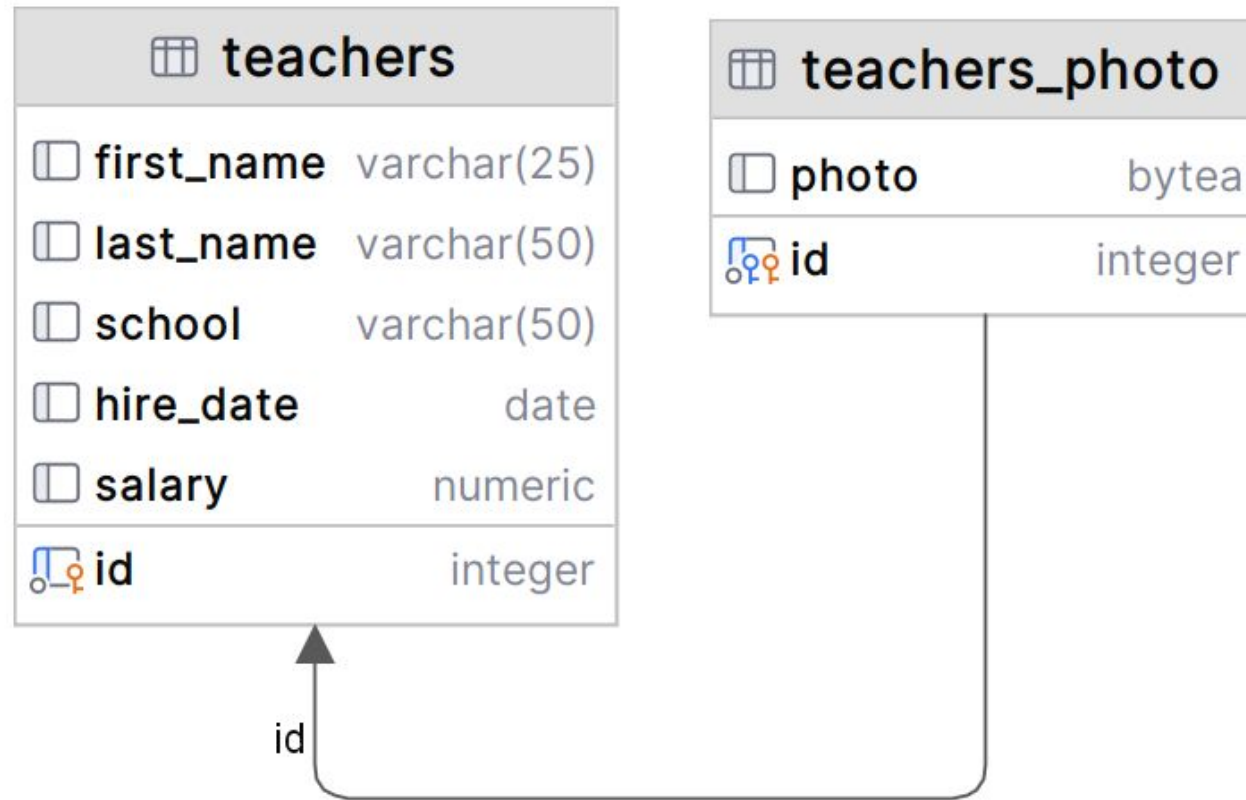
We will go into **table relationships** in **GREAT** detail when we start to create tables in the database.

One-to-many relationship



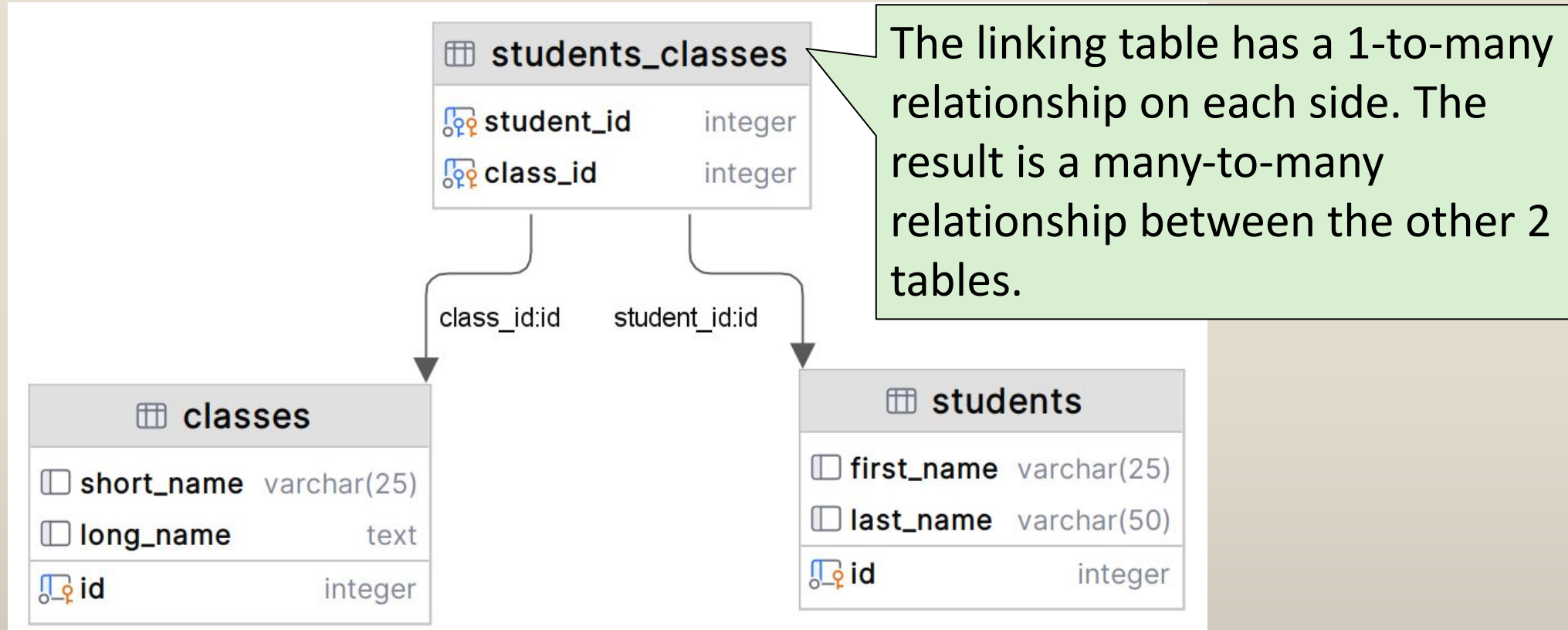
Each author can have zero or more books. The mapping is from one author to, potentially, many books.

One-to-one relationship



A teacher can have a photo. The photos are stored in a different table. A teacher cannot have more than 1 photo.

Many-to-many relationship

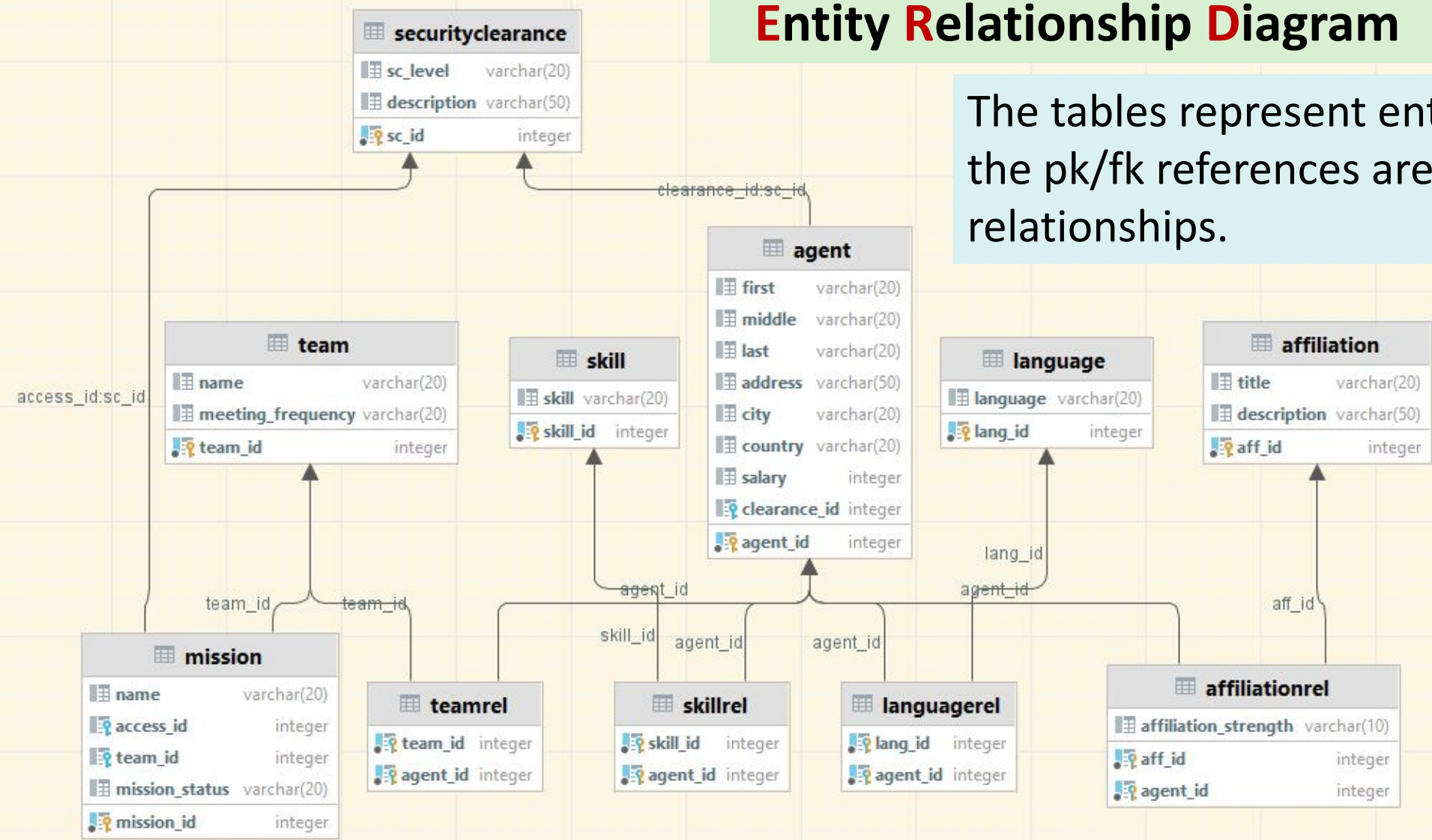


We have many **classes**. We have many **students**. A class can have zero or more students. A student can have zero or more classes.

We have a **linking table** between students and classes to represent the 1-to-many on each side.

Entity Relationship Diagram

The tables represent entities and the pk/fk references are the relationships.



Terms to know about columns

- **Data type** – columns have a type
- **Null value** – a cell whose value is set to nothing
- **Default value** – a value stored in a column when a row is inserted, but no value for that column has been provided.
- **Auto increment column** – a column whose value is controlled by the database and which is ***automatically*** incremented as new rows are inserted. PostgreSQL has a couple of ways of doing this. These are **often used for surrogate keys**.

A statement that creates a new table

CREATE TABLE **invoices**

The name of the new table

Column
names

invoice_id

vendor_id

invoice_number

invoice_date

invoice_total

payment_total

credit_total

terms_id

invoice_due_date

payment_date

SERIAL

INT

VARCHAR(50)

DATE

DECIMAL(9,2)

DECIMAL(9,2)

DECIMAL(9,2)

INT

DATE

DATE,

Column
types

PRIMARY KEY,

NOT NULL,

NOT NULL,

NOT NULL,

NOT NULL,

NOT NULL,

NOT NULL,

DEFAULT 0,

DEFAULT 0,

Default values

Constraints

CONSTRAINT invoices_fk_vendors

FOREIGN KEY (vendor_id)

REFERENCES vendors (vendor_id),

CONSTRAINT invoices_fk_terms

FOREIGN KEY (terms_id)

REFERENCES terms (terms_id)

More Constraints

A statement that adds a new column to a table

```
ALTER TABLE invoices  
ADD balance_due REAL;
```

A statement that drops the new column

```
ALTER TABLE invoices  
DROP COLUMN balance_due;
```

A statement that creates an index on the table

```
CREATE INDEX invoices_vendor_id_index  
ON invoices (vendor_id);
```

A statement that drops the new index

```
DROP INDEX invoices_vendor_id_index;
```

A SELECT statement that **retrieves**, **filters**, and **sorts** selected columns and rows

SELECT

Column names

invoice_number, invoice_date, invoice_total,
payment_total, credit_total,
invoice_total - payment_total - credit_total AS balance_due

The name
of the table

FROM invoices

WHERE invoice_total - payment_total - credit_total > 0

ORDER BY invoice_date ;

Sort criteria

The select criteria

The **result set** defined by the SELECT statement

	invoice_number	invoice_date	invoice_total	payment_total	credit_total	balance_due
▶	39104	2018-07-10	85.31	0.00	0.00	85.31
	963253264	2018-07-18	52.25	0.00	0.00	52.25
	31361833	2018-07-21	579.42	0.00	0.00	579.42
	263253268	2018-07-21	59.97	0.00	0.00	59.97
	263253270	2018-07-22	67.92	0.00	0.00	67.92

You'll hear me say "fetch data"

A SELECT statement that **joins** data

```
SELECT vendor_name, invoice_number, invoice_date, invoice_total
```

```
FROM vendors
```

```
INNER JOIN invoices
```

```
ON vendors.vendor_id = invoices.vendor_id
```

2 tables are joined by a foreign key relationship

```
WHERE invoice_total >= 500
```

```
ORDER BY vendor_name, invoice_total DESC ;
```

The **result set** defined by the SELECT statement

vendor_name	invoice_number	invoice_date	invoice_total
Federal Express Corporation	963253230	2018-07-07	739.20
Ford Motor Credit Company	9982771	2018-07-24	503.20
Franchise Tax Board	RTR-72-3662-X	2018-05-25	1600.00
Fresno County Tax Collector	P02-88D77S7	2018-05-03	856.92
IBM	Q545443	2018-06-09	1083.58
Ingram	31359783	2018-06-03	1575.00
Ingram	31361833	2018-07-21	579.42
Malloy Lithographing Inc	0-2058	2018-05-28	37966.19

A JOIN clause is used to **combine rows from two or more tables**, based on a related column/columns between them.

Terms to know about SQL

- **Query**
- **Result set** (result data) – the data returned from a `select` statement
- **Calculated value**
- **Joins**
 - **Inner join**
 - **Outer join** (multiple types of outer join)
 - **Cross join**

There are **5 ANSI joins**. We will go over those in great detail later in the course.

A statement that **adds** a new row into the Invoices table

```
INSERT INTO invoices
```

The name of the table
to receive the data.

```
(vendor_id, invoice_number, invoice_date,  
invoice_total, terms_id, invoice_due_date)
```

Column names

```
VALUES
```

```
(12, '3289175', '2018-07-18', 165, 3, '2018-08-17') ;
```

Data to be inserted

A statement that **modifies** the value of a column for one row

```
UPDATE invoices  
SET credit_total = 35.89  
WHERE invoice_number = '367447' ;
```

Only one row has an
invoice_number of '367447'

A statement that changes the value of a column for multiple rows

```
UPDATE invoices  
SET invoice_due_date  
= DATE_ADD(invoice_due_date, INTERVAL 30 DAY)  
WHERE terms_id = 4 ;
```

Multiple rows have a terms_id
column equal to 4.

A statement that **removes** data from the Invoices table

```
DELETE FROM invoices  
WHERE invoice_number = '4-342-8069' ;
```

Only one row has an
invoice_number of '4-342-8069'

A statement that **deletes** all paid invoices from the Invoices table

```
DELETE FROM invoices  
WHERE invoice_total - payment_total - credit_total = 0 ;
```

Multiple rows have a paid invoice.